COMPANION TO THE RAC ON KUBEADM LAB GUIDE

The golden image

Build one Oracle Linux 9.8 VM with everything the three lab nodes share, prove it with 20 checks, seal it and clone it. Or check and fix a golden image you already have.

`Oracle Linux 9.8 · UEK R7` `VirtualBox 7` `CRI-O 1.36 · kubeadm 1.36` `SELinux rac-ocne`

24 Oct 2026 · Follows Oracle's How to Install and Configure Oracle RAC on Oracle Cloud Native Environment. This guide covers its golden-image part (lab guide Phases 0 to 3.1).

`PowerShell`

on your PC

`golden VM`

root shell on a VM

How to use this guide

This guide builds the **golden image**: one Oracle Linux 9.8 VM, prepared once with everything the three lab nodes have in common, then sealed and cloned into `controlplane`, `worker1` and `worker2`. It covers the same ground as Phases 0 to 3.1 of the *Oracle RAC 19c on kubeadm* learning guide, as a document you can use on its own.

There are two ways through it:

- **Build a new golden image:** Steps 1 to 5, in order.
- **Check and fix a golden image you already have:** go to "Check and fix an existing golden image". It verifies every item from Step 3, fixes what's missing and seals the VM again. That's what was done to the lab's golden image on 24 Oct 2026.

Every step has the same parts: **What** it does, **Why** it's needed (what breaks without it), the **commands** to copy and paste, and a **Check** of what you should see.

Where each command runs

Every command block is labelled with where to paste it.

LABEL	WHERE	HOW TO GET THERE
`# PowerShell`	Windows PowerShell on the PC	Start menu, Windows PowerShell
`# golden VM`	root shell on the golden VM	`ssh -p 2205 root@127.0.0.1`

PowerShell is not bash. Blocks marked `# golden VM` are bash; paste them only after you've connected with `ssh`. A bash heredoc (`<<'EOF'`) pasted into PowerShell fails with "Missing file specification after redirection operator".

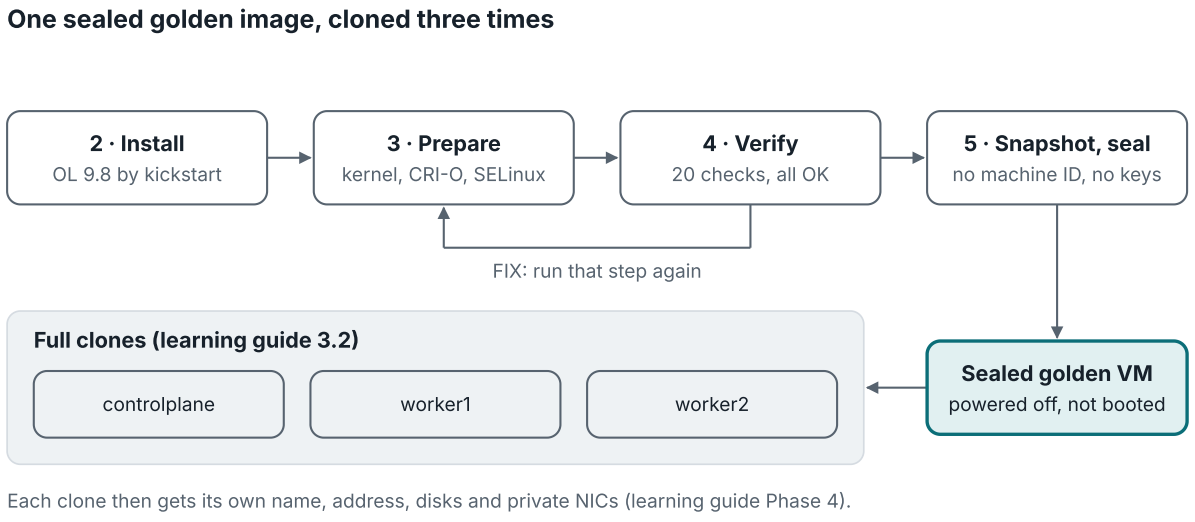
Lab values

ITEM	VALUE
Windows host	64 GB RAM, 8 threads, VirtualBox 7
VM folder	`D:\VMs\rac-poc`
Oracle Linux ISO	`D:\ISO images\OracleLinux-R9-U8-x86_64-dvd-20260618.iso`
SSH key	`%USERPROFILE%\.ssh\id_ed25519`
NAT Network `k8s-public`	10.0.1.0/24, DHCP off
Golden VM	`ol98-golden`, 10.0.1.5, SSH port 2205
Versions in the lab (24 Oct 2026)	UEK R7 5.15.0-325, CRI-O 1.36.7, Kubernetes 1.36.5

The root password is typed at a prompt in Step 2 and never written to a file you keep.

What the golden image contains

Why a golden image: every node needs the same kernel, packages, kernel settings and container runtime. Preparing them once and cloning three times is faster, and it guarantees the nodes match. RAC and Kubernetes both misbehave when nodes differ, and differences between nodes are hard to spot later.



Golden image lifecycle. A verify that says FIX loops back; only a verified image is sealed and cloned.

In the golden image (same on every node)

AREA	WHAT	WHY	STEP
VM hardware	2 vCPU, 4 GB, 50 GB OS disk, KVM paravirtualisation, NIC1 on `k8s-public`, NIC2/NIC3 on `rac-priv1` / `rac-priv2` with promiscuous mode Allow All, SATA ports fixed	Clones inherit it; workers are resized after cloning	2.1
OS	Oracle Linux 9.8 minimal, SELinux enforcing, firewall off, no swap partition, root SSH key	The doc runs SELinux enforcing; Kubernetes needs swap off	2.2
Kernel	Newest UEK R7 (5.15.0) as the default	The Oracle doc is tested on UEK R7	3.1
Kernel arguments	`transparent_hugepage=madvise clocksource=tsc`	THP `always` causes RAC node evictions; a stable clock for Grid	3.2
Swap	Off, and commented out in `/etc/fstab`	The kubelet refuses to run with swap on	3.3
Kernel modules	`overlay`, `br_netfilter` loaded at boot	Container layers and pod traffic through iptables	3.4
sysctls	`90-k8s.conf` (bridge, ip_forward), `91-oracle-rac.conf` (file-max, socket buffers, aio-max-nr)	Host-wide settings pods can't set themselves	3.4
Time and firewall	`chronyd` on, `firewalld` off	Grid and Kubernetes need the same time on every node	3.5
Tools	podman, unzip, jq, iproute-tc, tar, rsync, lsof, bash-completion, SELinux tools	podman builds the RAC image; unzip unpacks the Oracle homes	3.6
SELinux policy	Module `rac-ocne`; boolean `container_manage_cgroup on`	Oracle binaries in the pods; systemd inside a pod manages its cgroups	3.6
Kubernetes	Repos pinned to 1.36 with `exclude=`; `cri-o`, `kubelet`, `kubeadm`, `kubectl`, `cri-tools`; `/etc/crictl.yaml`	One runtime shared with podman, so the RAC image needs no registry	3.7
CRI-O config	`selinux = true`; pause image = kubeadm's; no sample bridge network; `crio` and `kubelet` enabled	SELinux labels for the policy module; no pause-image mismatch	3.8
Images	The 7 control-plane images pulled	Each clone starts with them in place	3.8
Names	`controlplane`, `worker1`, `worker2` in `/etc/hosts`; kubectl and crictl completion	Nodes resolve each other before DNS exists	3.9

Left for each clone (never in the golden image)

ITEM	WHY IT CAN'T BE IN THE GOLDEN IMAGE	WHERE IT'S DONE
Machine ID and SSH host keys	Must be unique; Kubernetes refuses duplicate machine IDs	Removed by the seal (Step 5); regenerated at the clone's first boot
Hardware UUID and MAC addresses	kubeadm checks `product_uuid`	VirtualBox, when it makes a full clone
Host name, IP address, kubelet node IP	Different per node	Learning guide Phase 4
CPU, memory, data disks	The control plane and the workers are sized differently	Learning guide 3.2
Private NICs up with no IP, promiscuous (workers)	Only workers run RAC pods	Learning guide 4.3 and 4.4, step c
`/scratch` and `/var/lib/containers` disks, doc directories, HugePages (workers)	Worker-only, and the disks only exist after cloning	Learning guide 4.3 and 4.4, steps d to g
Shared ASM disks	Attached to the two workers only	Learning guide 3.3
Kubernetes cluster membership	Created by `kubeadm init` and `join`	Learning guide Phase 5

STEP 1

Windows host and VirtualBox networks

Result of this step: an SSH key, an SSH configuration for the lab, and the VirtualBox networks every VM will use.

1.1 Check the host

Why: two workers of 22 GB, a 4 GB control plane and Windows itself need about 56 GB of the 64 GB. Worker vCPUs come from the host's thread count: with 8 threads, workers get 4 vCPUs and each RAC pod 3 CPUs. Giving 6 vCPUs to each worker on an 8-thread host overcommitted the CPU in the first build.

PowerShell

```
Get-CimInstance Win32_Processor | Select-Object NumberOfCores, NumberOfLogicalProcessors
[Math]::Round((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory / 1GB)
Get-PSDrive D | Select-Object Used, Free
```

Check: 8 or more logical processors, about 64 GB RAM, 250 GB or more free on D:. If you can, turn off Hyper-V, WSL2 and Memory Integrity so VirtualBox uses its own hypervisor; under Hyper-V it runs noticeably slower.

1.2 SSH key and lab SSH settings

Why: every VM gets this public key at install, so you log in without a password. All VMs are reached through `127.0.0.1` on different ports, and their host keys change when the golden image is sealed and cloned; the settings below stop SSH refusing those changes. They are for this lab only.

PowerShell

```
if (-not (Test-Path "$env:USERPROFILE\.ssh\id_ed25519")) { ssh-keygen -t ed25519 }   # press Enter at every prompt
$cfg = "$env:USERPROFILE\.ssh\config"
if (-not ((Test-Path $cfg) -and (Select-String -Path $cfg -Pattern '^Host 127\.0\.0\.1$' -Quiet))) {
@'

Host 127.0.0.1
    User root
    StrictHostKeyChecking no
    UserKnownHostsFile ~/.ssh/known_hosts_rac_lab
    LogLevel ERROR
    ServerAliveInterval 30
'@ | Add-Content -Encoding ascii $cfg
}
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub"
Select-String -Path $cfg -Pattern '^Host 127\.0\.0\.1$' -Context 0,5
```

The key is only created if you don't have one yet, and the settings are only added once, so the block is safe to run again.

Check: one `ssh-ed25519 ...` line (your public key), then the `Host 127.0.0.1` section with its five settings.

1.3 The VirtualBox networks

What: one NAT Network for the public side, SSH port forwards into it, and two internal networks for the RAC interconnect.

Why:

- `k8s-public` **(10.0.1.0/24):** every VM on one network with internet access through NAT. Kubernetes, Flannel's pod traffic and later the RAC public network all run here. DHCP is off because every node gets a fixed address; Kubernetes and Grid both depend on addresses not changing.
- **Port forwards:** the NAT Network can't be reached from Windows directly, so `127.0.0.1:2205/2210/2211/2212` are forwarded to SSH on each VM.
- `rac-priv1` **and** `rac-priv2` **:** two isolated internal networks for the RAC private interconnect (cache fusion and the cluster heartbeat). Two of them give Grid's HAIP redundancy. VirtualBox creates internal networks when a NIC is attached to them, so there is nothing to create here.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB natnetwork add --netname k8s-public --network "10.0.1.0/24" --enable --dhcp off
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-golden:tcp:[]:2205:[10.0.1.5]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-cp:tcp:[]:2210:[10.0.1.10]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-w1:tcp:[]:2211:[10.0.1.11]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-w2:tcp:[]:2212:[10.0.1.12]:22"
& $VB natnetwork start --netname k8s-public
& $VB natnetwork list
```

Check: `natnetwork list` shows `k8s-public`, `10.0.1.0/24`, DHCP disabled, and the four `ssh-*` rules.

STEP 2

Create the golden VM and install Oracle Linux 9.8

Result of this step: a VM called `ol98-golden` with a minimal Oracle Linux 9.8 install, reachable as `ssh -p 2205 root@127.0.0.1`.

2.1 Create the VM

Why each setting:

- **2 vCPU, 4 GB, 50 GB disk:** enough to install and prepare; each clone is resized later.
- `--paravirtprovider kvm`**:** Oracle Linux's UEK kernel runs best with KVM paravirtualisation (stable clock, lower overhead).
- **NIC1 on** `k8s-public`**; NIC2 and NIC3 on** `rac-priv1` **and** `rac-priv2` **with promiscuous mode** `allow-all`**.** RAC pods will put their own MAC addresses on these NICs through macvlan, and VirtualBox only delivers frames for unknown MACs in allow-all mode. Without it, the interconnect drops every packet.
- **Fixed SATA ports:** port 0 for the OS disk, ports 1 and 2 later for the worker data disks, 3 and 4 for the shared ASM disks, 5 for the DVD.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
$Base = "D:\VMs\rac-poc"; $Iso = "D:\ISO images\OracleLinux-R9-U8-x86_64-dvd-20260618.iso"; $g = "ol98-golden"
New-Item -ItemType Directory -Force -Path $Base | Out-Null
& $VB createvm --name $g --ostype Oracle_64 --basefolder $Base --register
& $VB modifyvm $g --cpus 2 --memory 4096 --vram 16 --graphicscontroller vmsvga --ioapic on --paravirtprovider kvm --rtcuseutc on --boot1 dvd --boot2 disk --boot3 none --boot4 none
& $VB modifyvm $g --nic1 natnetwork --nat-network1 k8s-public
& $VB modifyvm $g --nic2 intnet --intnet2 rac-priv1 --nicpromisc2 allow-all
& $VB modifyvm $g --nic3 intnet --intnet3 rac-priv2 --nicpromisc3 allow-all
& $VB storagectl $g --name SATA --add sata --controller IntelAhci --portcount 6 --hostiocache off
& $VB createmedium disk --filename "$Base\$g\$g-os.vdi" --size 51200 --variant Standard
& $VB storageattach $g --storagectl SATA --port 0 --device 0 --type hdd --medium "$Base\$g\$g-os.vdi"
& $VB storageattach $g --storagectl SATA --port 5 --device 0 --type dvddrive --medium $Iso
```

2.2 Kickstart and unattended install

What: a kickstart file answers every installer question, and `VBoxManage unattended install` boots the ISO with it.

Why these kickstart choices:

- **Static IP 10.0.1.5:** the temporary address of the golden VM. Each clone starts on it and moves to its own address in Phase 4 of the learning guide.
- **Your SSH key for root:** passwordless access from the start.
- **SELinux enforcing, firewall off:** the Oracle doc runs SELinux enforcing, with its own policy module for the RAC pods. The firewall is off to keep the lab simple (in production you would open the Kubernetes and Oracle ports instead).
- **One logical volume for `/`, no swap partition:** Kubernetes requires swap off.

The password is typed at a prompt and only exists in a temporary file that is deleted straight after.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Iso = "D:\ISO images\OracleLinux-R9-U8-x86_64-dvd-20260618.iso"; $g = "ol98-golden"
$sec = Read-Host "Root password for the lab VMs (8+ chars, no spaces or double quotes)" -AsSecureString
$pw  = [Runtime.InteropServices.Marshal]::PtrToStringAuto([Runtime.InteropServices.Marshal]::SecureStringToBSTR($sec))
$pub = (Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub" -Raw).Trim()
$ks = @"
text
cdrom
lang en_US.UTF-8
keyboard us
timezone America/New_York --utc
network --device=enp0s3 --bootproto=static --ip=10.0.1.5 --netmask=255.255.255.0 --gateway=10.0.1.1 --nameserver=8.8.8.8 --noipv6 --onboot=yes --activate
network --hostname=ol98-golden
rootpw --plaintext "$pw" --allow-ssh
sshkey --username=root "$pub"
selinux --enforcing
firewall --disabled
firstboot --disabled
skipx
ignoredisk --only-use=sda
zerombr
clearpart --all --initlabel --drives=sda
reqpart --add-boot
part pv.01 --size=1 --grow --ondisk=sda
volgroup ol pv.01
logvol / --vgname=ol --name=root --fstype=xfs --size=1 --grow
reboot --eject

%packages
@^minimal-environment
chrony
tar
rsync
%end
"@
$ksFile = "$env:TEMP\ol98-golden-ks.cfg"
[IO.File]::WriteAllText($ksFile, $ks.Replace("`r`n", "`n"), (New-Object Text.UTF8Encoding $false))
& $VB unattended install $g "--iso=$Iso" "--script-template=$ksFile" --hostname=ol98-golden.localdomain --user=rac "--password=$pw" --time-zone=America/New_York --locale=en_US --country=US --start-vm=headless
Remove-Item $ksFile; $pw = $null
```

The install takes 10 to 20 minutes. To watch it, open VirtualBox Manager, select `ol98-golden` and click **Show**.

Check (when SSH answers; until then you get "connection refused" or a timeout):

PowerShell

```
ssh -p 2205 root@127.0.0.1 "cat /etc/oracle-release; uname -r; ip -4 -br addr show enp0s3; swapon --show
| wc -l; getenforce"
```

You should see `Oracle Linux Server release 9.8`, `enp0s3 ... 10.0.1.5/24`, `0` swap devices and `Enforcing`. The kernel is still the installer's default here; Step 3 pins UEK R7.

STEP 3

OS prep for Kubernetes and RAC

Result of this step: the golden VM has everything all three nodes share: the UEK R7 kernel, the kernel settings Oracle and Kubernetes need, the SELinux policy for RAC pods, and CRI-O with kubeadm, kubelet and kubectl. Run every block in a root shell on the golden VM:

PowerShell

```
ssh -p 2205 root@127.0.0.1
```

3.1 Update the OS and pin the UEK R7 kernel

Why: the Oracle doc is tested on UEK R7 (5.15.0-208 or later). Oracle Linux 9.8 may default to UEK 8, so the newest UEK R7 kernel is installed and made the default. Running a kernel the doc wasn't tested on is a variable a PoC doesn't need.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf -y install dnf-plugins-core
dnf -y update
dnf config-manager --set-enabled ol9_UEKR7
dnf config-manager --set-disabled ol9_UEKR8 2>/dev/null || true
UEK7=$(dnf -q --disablerepo='*' --enablerepo=ol9_UEKR7 repoquery --latest-limit 1 --qf '%{version}-%
{release}' kernel-uek | tail -1)
echo "UEK R7: $UEK7"
dnf -y install "kernel-uek-${UEK7}"
grubby --set-default "/boot/vmlinuz-${UEK7}.x86_64"
```

3.2 Kernel arguments

Why:

- `transparent_hugepage=madvise`: Oracle recommends against THP `always`, which causes memory-access delays and, in RAC, node evictions. The doc asks for `madvise` on UEK R7.
- `clocksource=tsc`: a stable clock in a VM. Grid's cluster time and its heartbeat timing are sensitive to clock jumps.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
grubby --update-kernel=ALL --args="transparent_hugepage=madvise clocksource=tsc"
```

3.3 Swap off

Why: the kubelet refuses to run with swap on by default. Memory limits on pods only mean something when nothing is swapped out.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
swapoff -a
sed -ri '/\sswap\s/ s/^([^#])/#\1/' /etc/fstab
```

3.4 Kernel modules and sysctls

Why:

- `overlay` **and** `br_netfilter` **, plus the three** `net.*` **settings:** container image layers, and traffic between pods passing through iptables. Kubernetes networking needs them.
- **The Oracle settings:** the doc's values for the worker hosts (`fs.file-max` , socket buffers, `fs.aio-max-nr`). These are host-wide settings that pods can't set themselves. The settings pods *can* set for themselves (`kernel.shm*` , `kernel.sem`) come later, in the pod definitions.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
cat >/etc/modules-load.d/k8s.conf <<'EOF'
overlay
br_netfilter
EOF
modprobe overlay; modprobe br_netfilter
cat >/etc/sysctl.d/90-k8s.conf <<'EOF'
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
cat >/etc/sysctl.d/91-oracle-rac.conf <<'EOF'
fs.file-max = 6815744
net.core.rmem_default = 262144
net.core.rmem_max = 4194304
net.core.wmem_default = 262144
net.core.wmem_max = 1048576
fs.aio-max-nr = 1048576
EOF
sysctl --system >/dev/null
```

3.5 Time sync and firewall

Why: all nodes must agree on the time. Grid checks it, and Kubernetes certificates and leases depend on it. The firewall is off for the lab only.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf -y install chrony
systemctl enable --now chronyd
systemctl disable --now firewalld 2>/dev/null || true
```

3.6 Tools and the RAC SELinux policy

Why:

- **Tools:** `podman` builds the RAC image; `unzip` unpacks the Oracle software. It was missing in the first build, and the Grid unzip step failed until it was installed.
- **The `rac-ocne` policy module:** RAC pods run systemd as PID 1 in the SELinux domain `container_init_t`. Oracle binaries on `/u01` need to map and execute files, use text relocations and read the kernel log, which the default container policy forbids. This module is the doc's "Set up SELinux Module on Worker Nodes".
- `container_manage_cgroup`**:** systemd inside a pod must manage its own cgroups. Without this boolean, the RAC container exited about one second after starting, with no log output. OCNE sets it through its own SELinux package; on kubeadm you set it yourself.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf -y install container-selinux selinux-policy-devel policycoreutils-python-utils podman tar rsync unzip
jq iproute-tc bash-completion lsof
mkdir -p /var/opt && cd /var/opt
cat >rac-ocne.te <<'EOF'
module rac-ocne 1.0;

require {
        type container_init_t;
        type container_runtime_t;
        type container_file_t;
        type kernel_t;
        type bin_t;
        type lib_t;
        type textrel_shlib_t;
        class file { getattr map execute execmod setattr };
        class system syslog_read;
}

allow container_init_t container_runtime_t:file getattr;
allow container_init_t bin_t:file { map execute };
allow container_init_t container_file_t:file execmod;
allow container_init_t lib_t:file execmod;
allow container_init_t textrel_shlib_t:file setattr;
allow container_init_t kernel_t:system syslog_read;
EOF
make -f /usr/share/selinux/devel/Makefile rac-ocne.pp && semodule -i rac-ocne.pp
setsebool -P container_manage_cgroup on
cd ~
```

Check: `semodule -l | grep rac-ocne` prints `rac-ocne`, and `getsebool container_manage_cgroup` prints `on`.

3.7 Kubernetes and CRI-O packages

What: the Kubernetes v1.36 and CRI-O v1.36 repositories, then `cri-o`, `kubelet`, `kubeadm`, `kubectl` and `cri-tools`.

Why CRI-O: the doc builds the RAC image with podman and runs it as `localhost/oracle/database-rac`. CRI-O shares podman's image store, so a podman-built image is immediately visible to Kubernetes with no registry. With containerd you would have to import it separately. `cri-tools` gives `crictl`, which was also missing at first.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
cat >/etc/yum.repos.d/kubernetes.repo <<'EOF'
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
cat >/etc/yum.repos.d/cri-o.repo <<'EOF'
[cri-o]
name=CRI-O
baseurl=https://download.opensuse.org/repositories/isv:/cri-o:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://download.opensuse.org/repositories/isv:/cri-o:/stable:/v1.36/rpm/repodata/repomd.xml.key
EOF
dnf -y install cri-o kubelet kubeadm kubectl cri-tools --disableexcludes=kubernetes
echo "runtime-endpoint: unix:///var/run/crio/crio.sock" >/etc/crictl.yaml
```

The `exclude=` line stops a routine `dnf update` from upgrading Kubernetes by accident. Kubernetes is upgraded deliberately, one minor version at a time.

3.8 Configure CRI-O and pull the images

Why:

- `selinux = true` : containers get SELinux labels, which the `rac-ocne` module is written for.
- **The pause image:** set to the exact image kubeadm expects. Otherwise kubeadm warns about a mismatch, and two pause images get pulled.
- **No CRI-O sample bridge network:** Flannel and Multus provide the pod networks.
- **Images pulled now:** each clone then starts with the control-plane images already in place.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
mkdir -p /etc/crio/crio.conf.d
printf '[crio.runtime]\nselinux = true\n' >/etc/crio/crio.conf.d/90-rac-poc.conf
PAUSE_IMG=$(kubeadm config images list 2>/dev/null | grep '/pause:' | head -1)
printf '[crio.image]\npause_image = "%s"\n' "$PAUSE_IMG" >/etc/crio/crio.conf.d/91-pause.conf
rm -f /etc/cni/net.d/*crio*bridge* 2>/dev/null || true
systemctl enable --now crio
systemctl enable kubelet
kubeadm config images pull --cri-socket unix:///var/run/crio/crio.sock
```

The kubelet is enabled but won't start properly until `kubeadm init` or `join` gives it a configuration. That's expected.

3.9 Host names and shell completion

Why: every node can resolve the others by name before DNS exists. Completion saves typing.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
cat >>/etc/hosts <<'EOF'
10.0.1.10  controlplane
10.0.1.11  worker1
10.0.1.12  worker2
EOF
kubectl completion bash >/etc/bash_completion.d/kubectl
crictl completion bash >/etc/bash_completion.d/crictl
```

STEP 4

Verify the golden image

Result of this step: proof that the golden image has everything in "What the golden image contains", before you seal it.

4.1 Reboot

Why: the new kernel and kernel arguments only take effect after a reboot.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
systemctl reboot
```

Reconnect after a minute with `ssh -p 2205 root@127.0.0.1`.

4.2 Run the check

What: one check per item in the golden image. Each line prints `OK`, or `FIX` with the step to run again. It only reads; it changes nothing. The same block is used in "Check and fix an existing golden image".

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
c() { if eval "$2" >/dev/null 2>&1; then echo "OK    $1"; else echo "FIX   $1   -> rerun step $3"; fi; }
c "UEK R7 kernel is the default"        '[[ $(grubby --default-kernel) == *5.15.0*uek* ]]'
3.1
c "THP madvise + clocksource=tsc args"  'grubby --info=DEFAULT | grep -q
"transparent_hugepage=madvise.*clocksource=tsc\|clocksource=tsc.*transparent_hugepage=madvise"' 3.2
c "swap off in fstab"                   '! grep -Eq "^[^#].*\sswap\s" /etc/fstab'
3.3
c "overlay/br_netfilter at boot"        'grep -q br_netfilter /etc/modules-load.d/k8s.conf'
3.4
c "k8s sysctls"                         'grep -q ip_forward /etc/sysctl.d/90-k8s.conf'
3.4
c "Oracle sysctls"                      'grep -q aio-max-nr /etc/sysctl.d/91-oracle-rac.conf'
3.4
c "chronyd enabled"                     'systemctl is-enabled chronyd'
3.5
c "firewalld not enabled"               '! systemctl is-enabled firewalld'
3.5
c "tools (podman unzip jq tc lsof ...)" 'rpm -q container-selinux selinux-policy-devel policycoreutils-
python-utils podman tar rsync unzip jq iproute-tc bash-completion lsof' 3.6
c "SELinux module rac-ocne"             'semodule -l | grep -qw rac-ocne'
3.6
c "container_manage_cgroup on"          'getsebool container_manage_cgroup | grep -q "on$"'
3.6
c "k8s + CRI-O 1.36 packages"           'rpm -q cri-o kubelet kubeadm kubectl cri-tools && kubeadm
version -o short | grep -q "^v1\.36\."' 3.7
c "kubernetes repo exclude= line"       'grep -q "^exclude=kubelet" /etc/yum.repos.d/kubernetes.repo'
3.7
c "/etc/crictl.yaml"                    'grep -q crio.sock /etc/crictl.yaml'
3.7
c "CRI-O selinux = true"                'grep -q "selinux = true" /etc/crio/crio.conf.d/90-rac-poc.conf'
3.8
c "CRI-O pause image = kubeadm's"       'grep -q "$(kubeadm config images list 2>/dev/null | grep
/pause:)" /etc/crio/crio.conf.d/91-pause.conf' 3.8
c "crio + kubelet enabled"              'systemctl is-enabled crio && systemctl is-enabled kubelet'
3.8
c "control-plane images pulled"         '[ $(crictl images -q | wc -l) -ge 7 ]'
3.8
c "node names in /etc/hosts"            'grep -q "10.0.1.12 *worker2" /etc/hosts'
"3.9 (append only if missing)"
c "kubectl/crictl completion"           '[ -s /etc/bash_completion.d/kubectl ] && [ -s
/etc/bash_completion.d/crictl ]' 3.9
echo "--- evidence ---"
uname -r; cat /sys/kernel/mm/transparent_hugepage/enabled; cat
/sys/devices/system/clocksource/clocksource0/current_clocksource
getenforce; swapon --show | wc -l
rpm -q cri-o kubelet kubeadm kubectl cri-tools unzip podman
getsebool container_manage_cgroup
cat /etc/crio/crio.conf.d/91-pause.conf
```

```
systemctl is-active crio
crictl images
```

Check:

- all 20 lines say `OK`;
- the kernel is `5.15.0-...el9uek`, THP shows `[madvise]`, the clock source is `tsc`;
- SELinux is `Enforcing` and the swap count is `0`;
- every package has a version (none "not installed"), all on 1.36;
- `container_manage_cgroup --> on`;
- the pause image in `91-pause.conf` also appears in `crictl images`, and `crio` is `active`;
- `crictl images` lists 7 images: coredns, etcd, kube-apiserver, kube-controller-manager, kube-proxy, kube-scheduler and pause.

If a line says `FIX`, run the step it names again (all Step 3 blocks are safe to run twice, apart from the `/etc/hosts` append in 3.9), then run the check again.

STEP 5

Snapshot and seal

Result of this step: a powered-off, sealed golden VM, ready to clone, plus a snapshot of it from just before sealing.

Why:

- **The snapshot** (`pre-seal`) lets you go back and change the golden image without reinstalling.
- **Sealing** removes everything that must be unique per machine: the machine ID, the SSH host keys, the logs and the shell history. Kubernetes refuses to join two nodes with the same machine ID, and identical SSH host keys would make every node look the same to SSH.
- **VirtualBox gives each full clone a new hardware UUID and new MAC addresses**, which kubeadm also checks.

5.1 Snapshot

`golden VM`

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
systemctl poweroff
```

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $g = "ol98-golden"
while ("$(& $VB showvminfo $g --machinereadable | Select-String '^VMState=')" -notmatch 'poweroff') {
Start-Sleep 3 }
& $VB storageattach $g --storagectl SATA --port 5 --device 0 --type dvddrive --medium emptydrive
& $VB snapshot $g take pre-seal --description "Prepared golden image before sealing"
& $VB startvm $g --type headless
```

If `startvm` says the VM is locked, wait 10 seconds and run it again; VirtualBox holds a lock briefly after a snapshot. If a snapshot called `pre-seal` already exists, use a new name such as `pre-seal-2`.

5.2 Seal

Reconnect with `ssh -p 2205 root@127.0.0.1`, then:

golden VM

```
(the last command powers it off; don't start it again before cloning)
CONNECT FIRST  ssh -p 2205 root@127.0.0.1

dnf clean all
rm -f /etc/ssh/ssh_host_*
truncate -s 0 /etc/machine-id
rm -f /var/lib/dbus/machine-id
rm -rf /tmp/* /var/tmp/*
journalctl --rotate; journalctl --vacuum-time=1s
rm -f /root/.bash_history; history -c
poweroff
```

Check:

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $g = "ol98-golden"
& $VB list runningvms
& $VB snapshot $g list
```

`ol98-golden` is not running, and the snapshot list shows your `pre-seal` snapshot.

Don't boot the golden VM again before cloning. Booting it gives it a new machine ID and new SSH host keys, so it's no longer sealed. If you do boot it, run 5.2 again before cloning.

Check and fix an existing golden image

When to use this: you already have a golden VM, built earlier or by an older version of the scripts, and want to be sure it has everything in "What the golden image contains" before cloning from it again.

Why it matters: during the first build, four problems were fixed by hand on the running nodes after cloning, and only later added to the golden-image preparation. A golden image built before those fixes still lacks them, and every clone inherits the gap:

ISSUE	MISSING FROM AN OLDER GOLDEN IMAGE	WHAT BREAKS
#8	`cri-tools` and `/etc/crictl.yaml`	`crictl: command not found`
#9	CRI-O pause image set to kubeadm's	a kubeadm warning, and two pause images pulled
#17, #44	SELinux `container_manage_cgroup on`	RAC pods exit about one second after starting, with no logs, and loop in BackOff
#20	`unzip`	the Grid gold image fails to unpack

Start and connect

Why power off the cluster first: the two workers and the control plane hold about 48 GB of RAM. Skip that line if they're already off.

```powershell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
foreach ($v in "worker1","worker2","controlplane") { & $VB controlvm $v acpipowerbutton 2>$null }
& $VB startvm ol98-golden --type headless
```

A sealed golden VM makes new SSH host keys when it boots, so forget the old one first. After about a minute:

```powershell
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

Answer `yes` to the host key question.

Check

Run the check from 4.2. Every `FIX` line names the step that sets that item.

Fix

Run the steps the `FIX` lines name. The block below runs 3.6, 3.7 and 3.8 plus the completion lines from 3.9 in one go, which covers every gap found on 24 Oct 2026. All of it is safe to run on a golden VM that already has some of these items. It leaves out the `/etc/hosts` append from 3.9, which would add the names a second time; run that line only if the check said `FIX` for it.

It has one line that the step itself doesn't: `systemctl restart crio`. CRI-O is already running on an existing golden VM, so it has to restart to pick up the pause-image setting. On a new build, CRI-O starts for the first time in 3.8.

If the check also said `FIX` for 3.1 or 3.2 (kernel or kernel arguments), run those steps too and reboot (4.1).

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
# 3.6 tools, SELinux module (reinstalls the same module), cgroup boolean
dnf -y install container-selinux selinux-policy-devel policycoreutils-python-utils podman tar rsync unzip
jq iproute-tc bash-completion lsof
mkdir -p /var/opt && cd /var/opt
cat >rac-ocne.te <<'EOF'
module rac-ocne 1.0;

require {
        type container_init_t;
        type container_runtime_t;
        type container_file_t;
        type kernel_t;
        type bin_t;
        type lib_t;
        type textrel_shlib_t;
        class file { getattr map execute execmod setattr };
        class system syslog_read;
}

allow container_init_t container_runtime_t:file getattr;
allow container_init_t bin_t:file { map execute };
allow container_init_t container_file_t:file execmod;
allow container_init_t lib_t:file execmod;
allow container_init_t textrel_shlib_t:file setattr;
allow container_init_t kernel_t:system syslog_read;
EOF
make -f /usr/share/selinux/devel/Makefile rac-ocne.pp && semodule -i rac-ocne.pp
setsebool -P container_manage_cgroup on
cd ~
# 3.7 repositories (pinned to 1.36), Kubernetes and CRI-O packages
cat >/etc/yum.repos.d/kubernetes.repo <<'EOF'
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
cat >/etc/yum.repos.d/cri-o.repo <<'EOF'
[cri-o]
name=CRI-O
baseurl=https://download.opensuse.org/repositories/isv:/cri-o:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://download.opensuse.org/repositories/isv:/cri-o:/stable:/v1.36/rpm/repodata/repomd.xml.key
EOF
dnf -y install cri-o kubelet kubeadm kubectl cri-tools --disableexcludes=kubernetes
echo "runtime-endpoint: unix:///var/run/crio/crio.sock" >/etc/crictl.yaml
```

```
# 3.8 CRI-O config, pause image, images (with a restart, because CRI-O is already running)
mkdir -p /etc/crio/crio.conf.d
printf '[crio.runtime]\nselinux = true\n' >/etc/crio/crio.conf.d/90-rac-poc.conf
PAUSE_IMG=$(kubeadm config images list 2>/dev/null | grep '/pause:' | head -1)
printf '[crio.image]\npause_image = "%s"\n' "$PAUSE_IMG" >/etc/crio/crio.conf.d/91-pause.conf
rm -f /etc/cni/net.d/*crio*bridge* 2>/dev/null || true
systemctl enable --now crio
systemctl restart crio
systemctl enable kubelet
kubeadm config images pull --cri-socket unix:///var/run/crio/crio.sock
# 3.9 completion only (the node names are normally already in /etc/hosts)
kubectl completion bash >/etc/bash_completion.d/kubectl
crictl completion bash >/etc/bash_completion.d/crictl
```

It takes 3 to 5 minutes, mostly the image pull. `dnf` may update CRI-O or the Kubernetes packages to a newer 1.36 release; the repositories are pinned to 1.36, so the minor version can't change.

Verify again, then seal

1. Run the check from 4.2 again. Every line should now say `OK`.
2. Seal the VM again with Step 5. Name the snapshot `pre-seal-2` (or the next free number), because `pre-seal` already exists.

Result on the lab's golden image, 24 Oct 2026

CHECK	BEFORE	AFTER
Kernel, kernel arguments, swap, modules, sysctls, chrony, firewall	OK	OK
SELinux module `rac-ocne`, kubernetes repo `exclude=`, `/etc/crictl.yaml`, CRI-O `selinux = true`, services enabled, `/etc/hosts` names	OK	OK
Tools (3.6)	FIX	OK, `unzip-6.0-60.el9`, `podman-5.8.2`
`container_manage_cgroup` (3.6)	FIX	OK, `on`
Kubernetes and CRI-O packages (3.7)	FIX	OK, CRI-O 1.36.7, kubelet/kubeadm/kubectl 1.36.5, cri-tools 1.36.0
Pause image (3.8)	FIX	OK, `registry.k8s.io/pause:3.10.2`
Control-plane images (3.8)	FIX	OK, 7 images
kubectl and crictl completion (3.9)	FIX	OK

The kernel was already `5.15.0-325.221.6.5.el9uek`. The before-fix output didn't record which individual tool or package was missing; the issue log points to `unzip` (#20) and `cri-tools` (#8).

Next: clone the three nodes

The sealed golden VM is the starting point for the *Oracle RAC 19c on kubeadm* learning guide from **3.2 Clone and size the nodes** onwards:

1. **3.2:** full clones `controlplane`, `worker1` and `worker2`, then size them (workers 4 vCPU and 22 GB, with an 80 GB and a 100 GB data disk).
2. **3.3:** the two shared ASM disks, attached to both workers.
3. **Phase 4:** personalize each clone, one at a time, because every clone boots on the golden VM's address 10.0.1.5.
4. **Phase 5 onwards:** Kubernetes, Multus, the ASM volumes, the RAC image, the RAC pods, Grid and the database.

Before cloning, check:

- `ol98-golden` is powered off and hasn't been booted since 5.2;
- the NAT Network `k8s-public` exists (`VBoxManage natnetwork list`), because the clones inherit NIC1's attachment to it by name;
- no VMs called `controlplane`, `worker1` or `worker2` exist yet (`VBoxManage list vms`).

To change the golden image later: boot it, make the change, add a check for it to 4.2, run the check, then seal again with a new snapshot name. Clones made earlier don't pick up the change; apply it to them separately or reclone.

Built from the lab's build log and scripts (rac-k8s-poc). Values are from the author's lab; change them to match yours.